SPACE X FALCON 9 LANDING PREDICTION

Data Science Capstone Report
Corrected submission draft

Research focus
Can mission parameters help predict successful Falcon 9 first-stage recovery?

Executive Summary

Business context, scope, and the results reported in the submitted analysis

90	88.9%	4
launch records reported	Decision Tree test accuracy reported	classification models compared

Problem

Falcon 9 first-stage recovery affects launch economics because successful recovery can enable booster reuse. The project frames landing outcome as a binary classification task using mission and vehicle features.

Approach

The submitted report describes SpaceX API data collection, historical table scraping, data cleaning, exploratory analysis, SQL queries, Folium mapping, a Plotly Dash dashboard, and comparison of Logistic Regression, SVM, Decision Tree, and KNN models.

Result and interpretation

The source report states that the Decision Tree reached 88.9% holdout accuracy. This figure should be presented as a reported result until the underlying notebook, data split, and evaluation output are linked and reproducible. Accuracy alone does not establish operational readiness.

Correction applied: removed unsupported wording that claimed the results “prove” landing outcomes are highly predictable or establish commercial market dominance. Those conclusions require reproducible analysis and external evidence.

Introduction and Business Problem

Research question, objectives, and stakeholders

Business problem

Launch customers and analysts may benefit from a structured estimate of first-stage recovery probability. A failed recovery and a successful recovery have different implications for refurbishment planning, risk assessment, and mission economics.

Research question

Can the available mission-level variables—such as payload mass, orbit, launch site, flight sequence, and booster characteristics—help classify whether a Falcon 9 first stage lands successfully?

Project objectives

- Describe the dataset and document how the target outcome is defined.
- Explore how landing outcomes vary across time, orbit, payload, and launch site.
- Use SQL to summarize launch records and compare site-level aggregates.
- Present launch-site geography using a Folium map and interactive filtering using Plotly Dash.
- Compare baseline classification models and report metrics with limitations.

Intended users: data analysts, launch-market researchers, satellite operators, and educators. The model is an exploratory capstone tool, not a flight-safety system or a substitute for mission-specific engineering analysis.

Data Collection Methodology

Source provenance and integration workflow

Source / step	Purpose	Required validation
SpaceX REST API	Retrieve launch metadata, dates, payload and rocket/booster fields, launch site, orbit, and outcome-related fields.	Record API endpoint/version, retrieval date, fields used, and any pagination or filtering.
Historical launch table scraping	Supplement or cross-check historical mission records where needed.	Document exact page URL, retrieval date, table structure, and parsing assumptions.
Integration	Standardize column names, dates, categories, and identifiers; deduplicate records.	Match records by stable launch/flight identifier; report duplicate and unmatched counts.
Analysis dataset	Create a documented table for EDA, SQL, maps, and machine learning.	State final row count, date range, target class counts, and excluded records.

The submitted report describes 90 launch records and 17 operational attributes. These counts should be confirmed against the final saved dataset and code before being treated as verified.

Repository links: the source PDF contains example.com URLs, which are placeholders rather than a verifiable GitHub project. The actual repository address is not present in the provided materials, so it has not been invented here.

Data Wrangling and Feature Engineering

Target definition, missing values, and categorical encoding

Target variable definition

Raw outcome category (as described in source)	Binary class	Interpretation
True ASDS / True RTLS	1	Successful first-stage recovery
False ASDS / False RTLS	0	Attempted recovery was unsuccessful
None / no recovery attempt	Requires explicit rule	Do not automatically equate no attempt with a failed landing unless this is the project's stated prediction target.

Preprocessing steps

- Inspect missingness by column and determine whether each missing value means unknown, not applicable, or no event.
- If imputing payload mass, calculate the imputation value from training data only to avoid leakage; compare mean and median where appropriate.
- One-hot encode categorical predictors such as orbit and launch site. Fit preprocessing inside a pipeline so the holdout set does not influence training.
- Exclude fields that directly reveal the outcome or would not be known at prediction time.
- Check class balance and use stratified train/test splitting where appropriate.

Correction applied: the original report states five missing payload masses were filled with a mean of 6,123.5 kg and that 12 columns expanded to 72 dummy variables. These exact values need to be checked against the code; the revised report treats them as unverified source claims, not confirmed facts.

Exploratory Data Analysis and Visualization

Questions, plots, and analytical checks

Analysis question	Recommended visualization	What to report
Did recovery outcomes change over time?	Annual launch counts and annual success-rate line chart.	Show denominators and confidence intervals where feasible; avoid claiming a trend from sparse years.
Does outcome vary by orbit?	Orbit-level counts plus success-rate bars.	Show count per orbit; flag small categories rather than presenting 100% rates as definitive.
How does payload relate to outcome?	Payload mass scatter/box plot split by target class.	Report overlap and distribution, not just a single threshold.
Do sites differ?	Launch-site counts and rates.	Keep site names standardized and separate launch count from success rate.
Are predictors redundant?	Correlation heatmap for numeric features and categorical summaries.	Avoid interpreting correlation as causation.

The submitted report describes Matplotlib/Seaborn visualizations and Pandas aggregations. To make this section assessable, the final submission should include actual plots with titles, axis labels, legends, source/units, and a short interpretation beneath each chart.

Quality check: calculate each rate as successful outcomes divided by all eligible records in that group. Show the numerator and denominator or group size to make results auditable.

Predictive Analysis: Classification Methodology

Model pipeline and fair evaluation

Recommended reproducible workflow

• Define the target precisely and document which records are included.

• Split the dataset into training and holdout test sets before fitting imputers, scalers, or encoders.

• Use a scikit-learn Pipeline/ColumnTransformer to prevent preprocessing leakage.

• Tune model hyperparameters using cross-validation on training data only.

• Evaluate the selected model once on the holdout test set and retain the test set for final reporting.

Model	Role in comparison	Notes
Logistic Regression	Interpretable linear baseline	Scale numeric predictors; inspect class-specific precision and recall.
Support Vector Machine	Nonlinear or linear margin-based model	Tune kernel and C; scale numeric features.
Decision Tree	Rule-based nonlinear model	Control depth and leaf size to reduce overfitting.
K-Nearest Neighbors	Distance-based baseline	Scale numeric predictors and tune neighbor count.

The original report says the data were split 80/20 (72 train, 18 test) and that 10-fold GridSearchCV was used. With a small dataset, 10-fold validation can be unstable; document the scoring metric, stratification, random seed, and actual cross-validation results.

EDA with Visualization: Results and Interpretation

Separate observed results from hypotheses

The submitted report lists these patterns, but the provided materials do not include the underlying chart images or executable dataset. They should therefore be treated as reported observations requiring verification.

Claim in submitted report	How to validate before final submission
Annual landing success improved over time and exceeded 80% by 2017.	Plot yearly successful / eligible launches and annotate counts. Confirm year filters and outcome definition.
GTO outcomes were lower than LEO/SSO outcomes.	Show orbit-level counts and rates; investigate class size and mission mix.
Some orbit categories had 100% success.	Display denominators; avoid strong claims for small samples.
Payloads between 2,000 and 5,500 kg showed higher success.	Use a plot or model-based analysis; avoid choosing a range after looking at outcomes without explaining the exploratory nature.
Launch-site success rates differed.	Standardize site labels and compute rates from raw counts; resolve duplicated/overlapping names.

Required visual evidence: at least one time-trend plot, one orbit comparison, and one payload/outcome plot. Each should include a concise finding and a limitation.

EDA with SQL Queries and Results

Query logic, data quality, and consistent aggregates

Example SQL pattern for site-level summaries (adjust column names to match the actual SQLite schema):

```
SELECT
Launch_Site,
COUNT(*) AS Total_Launches,
SUM(CASE WHEN Landing_Outcome IN ('True ASDS','True RTLS')
THEN 1 ELSE 0 END) AS Successful_Landings,
100.0 * SUM(CASE WHEN Landing_Outcome IN ('True ASDS','True RTLS')
THEN 1 ELSE 0 END) / COUNT(*) AS Success_Rate_Pct,
AVG(Payload_Mass_Kg) AS Avg_Payload_Kg
FROM SPACEXTBL
GROUP BY Launch_Site
ORDER BY Success_Rate_Pct DESC;
```

The source table lists site counts of 26, 22, 10, and 32 (90 total), with reported successes of 15, 17, 8, and 21. These imply rates of 57.7%, 77.3%, 80.0%, and 65.6% respectively, but the site labels include both “CCAFS LC-40” and “CCAFS SLC-40.” Verify whether these are genuinely distinct labels or a naming inconsistency before interpreting site differences.

Important: do not order by successful-landings count and then describe the result as highest success rate. Those are different measures. The query above orders by rate and makes the target mapping explicit.

Folium Map and Spatial Analysis

Show launch-site locations without unsupported distance claims

Map content

Use Folium markers for each launch site, with pop-ups containing the standardized site name and launch count. If location coordinates are available in the dataset or a documented source, add a layer control and a legend.

Distance calculation

If measuring distance to coastline, rail, or road infrastructure, document the coordinate source, distance method, and exact reference point. Straight-line distance is not the same as road/rail travel distance.

Correction applied

The original report states that all pads are within 1 km of coastlines, and gives example distances such as 0.85 km to the coast and 1.2 km to rail. The provided materials do not include map output, coordinates, or calculation code to verify those numbers. They should be removed or supported by reproducible calculations before submission.

Assessment evidence to include: a screenshot or embedded map, readable site markers, source attribution, and a short explanation of what spatial proximity does—and does not—tell us about recovery operations.

Plotly Dash: Application and User Interaction

Describe controls, outputs, and limits

Dashboard element	Expected behavior	Evidence to include
Launch-site dropdown	Filter the records to one site or show all sites.	Screenshot of control and resulting chart.
Payload range slider	Filter records by selected payload-mass interval.	State slider units and confirm filtered records update.
Outcome chart	Compare success/failure counts or rates for filtered records.	Label classes, counts, and denominator.
Payload scatter plot	Show payload mass against outcome, optionally colored by booster generation.	Add axis labels, legend, and a brief interpretation.
Data summary	Show number of records remaining after filters.	Prevents percentages from being read without sample size.

The original report describes these controls but does not provide a working application URL that can be verified. The listed GitHub address is a placeholder. Include the real repository link and, if deployed, the actual application URL; otherwise label it as a local demonstration.

Correction applied: avoid saying the dashboard is “real-time” unless it receives live updates. Interactive filtering of a static dataset is not real-time telemetry.

Model Comparison and Performance

Reported results, presented transparently

The following figures are copied from the submitted report and have not been independently reproduced. Keep them only if they match the saved notebook output.

Model	Reported CV score	Reported test accuracy	Validation status
Logistic Regression	84.6%	83.3%	Verify notebook output
Support Vector Machine	84.6%	83.3%	Verify kernel and scoring metric
Decision Tree	88.9%	88.9%	Verify selected parameters and holdout predictions
K-Nearest Neighbors	84.6%	83.3%	Verify scaling and selected k

Interpretation

Within the submitted comparison, the Decision Tree has the highest reported holdout accuracy. Accuracy should be supplemented by precision, recall, F1-score, and a confusion matrix, particularly if the classes are imbalanced or the cost of false predictions differs.

Correction applied: the source report says cross-validation and holdout accuracy “aligned perfectly” and that the model did so “without overfitting.” Similar scores alone do not prove absence of overfitting. Use cautious language and include reproducible validation evidence.

Confusion Matrix and Model Limitations

Use the same test set and consistent class definitions

Reported Decision Tree confusion matrix (from the submitted PDF):

	Predicted failure (0)	Predicted success (1)
Actual failure (0)	TN = 4	FP = 2
Actual success (1)	FN = 0	TP = 12

These counts sum to 18 test records and imply 16/18 = 88.9% accuracy. For the positive class, recall is 12/(12+0) = 100%; precision is 12/(12+2) = 85.7%. These calculations follow from the confusion matrix as printed and are not independent verification of the underlying model.

What the errors mean

• False positive: model predicts recovery, but the observed outcome is failure. This can create overconfidence if used for planning.

• False negative: model predicts failure, but recovery succeeds. This may undervalue a reusable outcome.

• A small holdout set makes each error materially affect the reported percentages.

• Use the same test split when comparing models and confirm the confusion matrix matches the accuracy table.

Correction applied: replace the phrase “ensuring no successful recovery missions were misclassified” with “no false negatives were observed in this 18-record test set.” The latter accurately limits the claim to the sample.

Conclusion and Strategic Recommendations

Answer the research question and state limitations

Conclusion

The submitted analysis demonstrates a complete project structure spanning data acquisition, preprocessing, exploratory analysis, SQL, geospatial mapping, dashboard design, and supervised classification. The reported Decision Tree accuracy is promising as a capstone result, but its reliability depends on validating the dataset, target definition, preprocessing pipeline, and evaluation outputs.

Recommendations

• Publish the actual repository URL and ensure all referenced notebooks and scripts are accessible.

• Re-run the analysis from a clean environment and align every number in the PDF with saved outputs.

• Standardize launch-site names and verify all grouped counts and success-rate denominators.

• Report precision, recall, F1-score, confusion matrix, class distribution, and baseline comparison.

• Describe the model as an exploratory decision-support aid—not as a validated commercial pricing, insurance, or flight-safety system.

• Include a clear limitations section covering small sample size, potential data leakage, changing mission mix, and the difference between historical association and causation.

Business relevance

A reliable recovery prediction workflow could support scenario analysis and risk discussion. However, specific pricing savings, booster turnaround targets, or market-share claims should not be attributed to this model unless independently sourced and directly supported by the analysis.

Innovative Insights and Final Quality Check

Evidence that strengthens the assessment

Potential innovative insights to test (do not present as confirmed findings until computed):

• Compare model performance across early and later launch eras to test whether the relationship between features and recovery changes over time.

• Measure how much the model's performance changes when orbit or launch site is removed, using a consistent evaluation design.

• Compare success rates with counts and uncertainty intervals to avoid overstating small categories.

• Use feature importance or permutation importance to explain model behavior, while noting that importance is not causation.

• Check whether the same site/outcome patterns persist after standardizing labels and removing duplicate records.

Before submitting, confirm each item:

■	Replace example.com repository URLs with the real public GitHub URL.
■	Ensure the repository contains completed notebooks, SQL script, Dash app, and map code actually referenced in the report.
■	Embed actual EDA plots, SQL result output, Folium map, and dashboard screenshots.
■	Reconcile all counts, rates, metrics, dates, and site names against code output.
■	Re-run the model and verify test split, cross-validation, confusion matrix, and metrics.
■	Add sources for any launch cost, fairing cost, market-share, or booster amortization claims—or remove them.
■	Export the final presentation as PDF and open it to check page layout and legibility.

Known item still requiring your input: the real GitHub repository URL was not provided. This draft intentionally flags that issue rather than retaining the invalid example.com link or inventing a repository.